

Realizzazione della Macchina Astratta

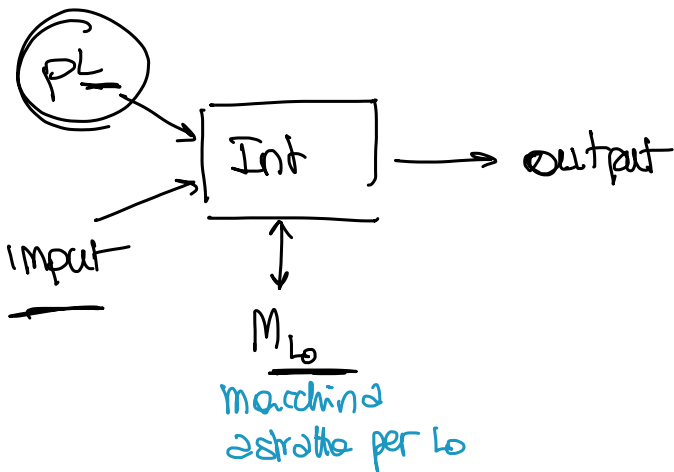
Soluzione Interpretativa

Soluzione Compilativa

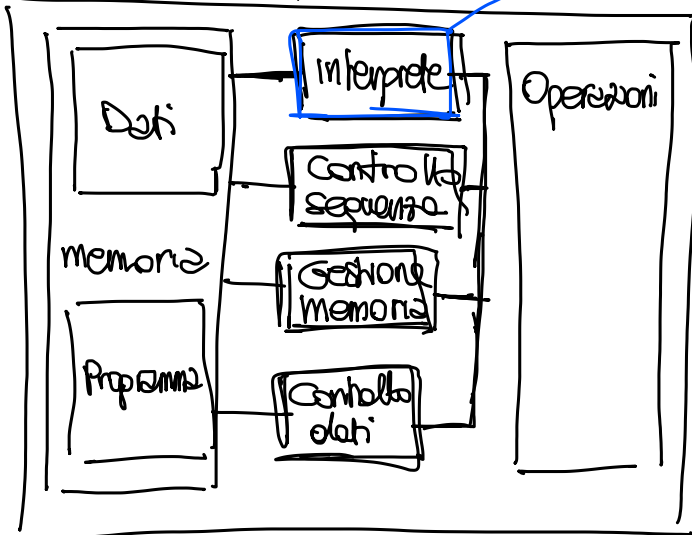
Interprete da L a L₀

$$I^{L_0}(P^L, \text{input}) = \text{output}$$

$$I^{L_0} : \text{Prog}^L \times D \rightarrow D$$



Macchina astratta



begin

go := true;

while go do begin

FETCH (OPCODE, OPINFO) at PC

DECODE (OPCODE, OPINFO)

if OPCODE needs ARGS

then FETCH (ARGS)

CASE OPCODE of

OP₁ : EXECUTE (OP₁, ARGS);

OP₂ : EXECUTE (OP₂, ARGS);

⋮

OP_M : EXECUTE (OP_M, ARGS);

HLT : go := false;

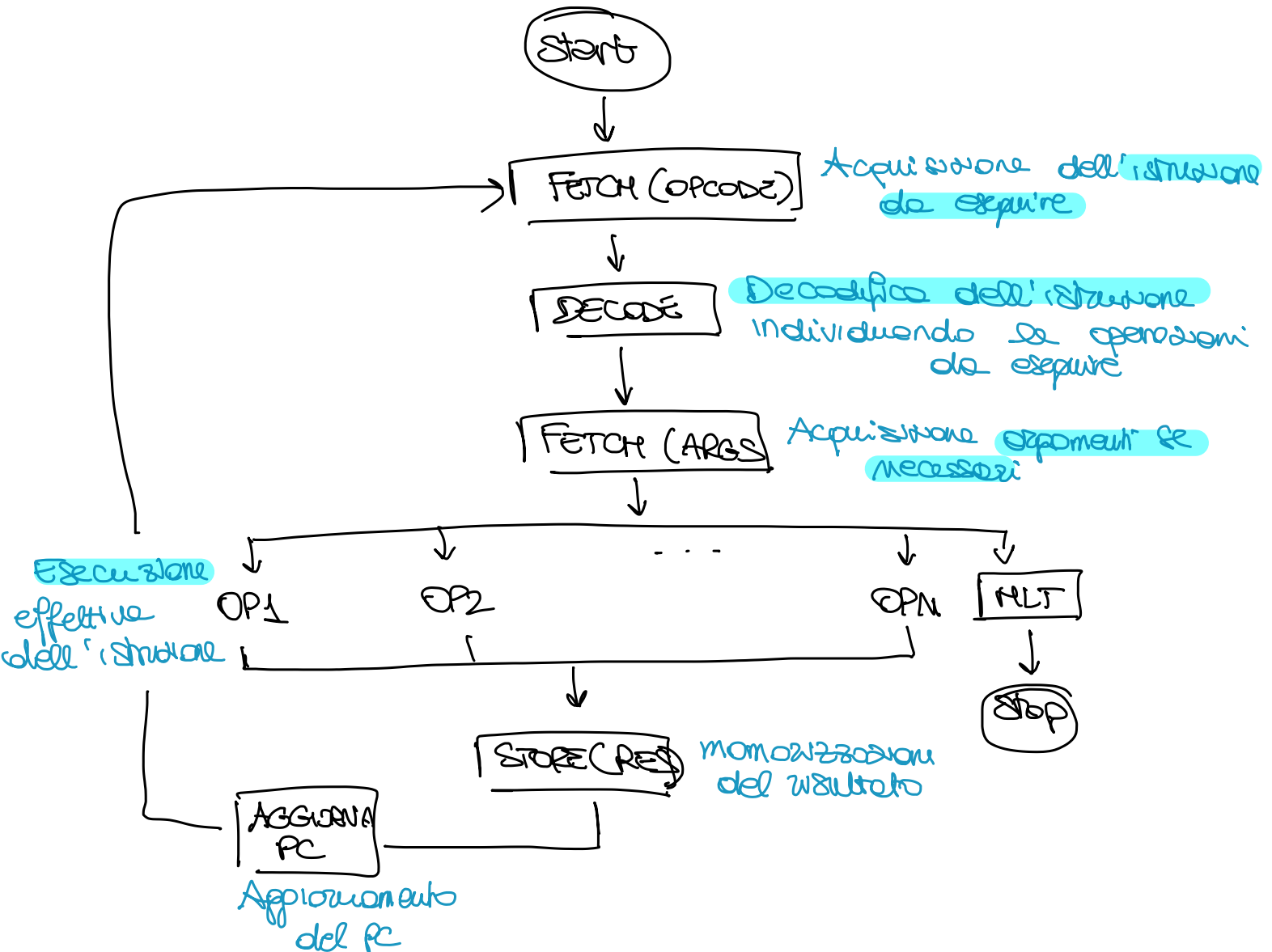
if OPCODE has result then

STORE (RES)

controllo dati

program counter

$PC := PC + \text{SIZE}(\text{OPCODE})$
 ← Controllo sequenza
 end
 end



SOLUZIONE COMPILATIVA

$L \rightarrow$ **Macchina Astratta = compilatore**

COMPILATORE: Un compilatore da L a L_0
 (ling. sorgente → ling. target)

TRADUTTORE è un programma $C^{L_0}: \text{Prog}^L \rightarrow \text{Prog}^{L_0}$

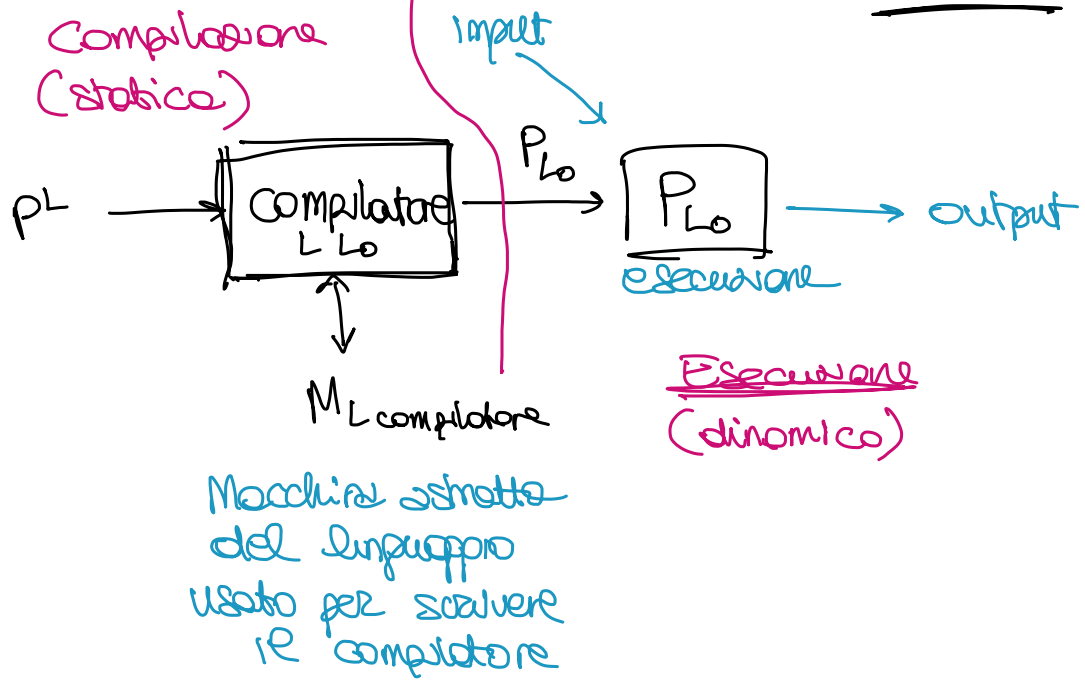
$C^{L_0}(P^L) = P^{L_0}$ tale che $\forall d \in D$

$P^L(d) = P^{L_0}(d)$

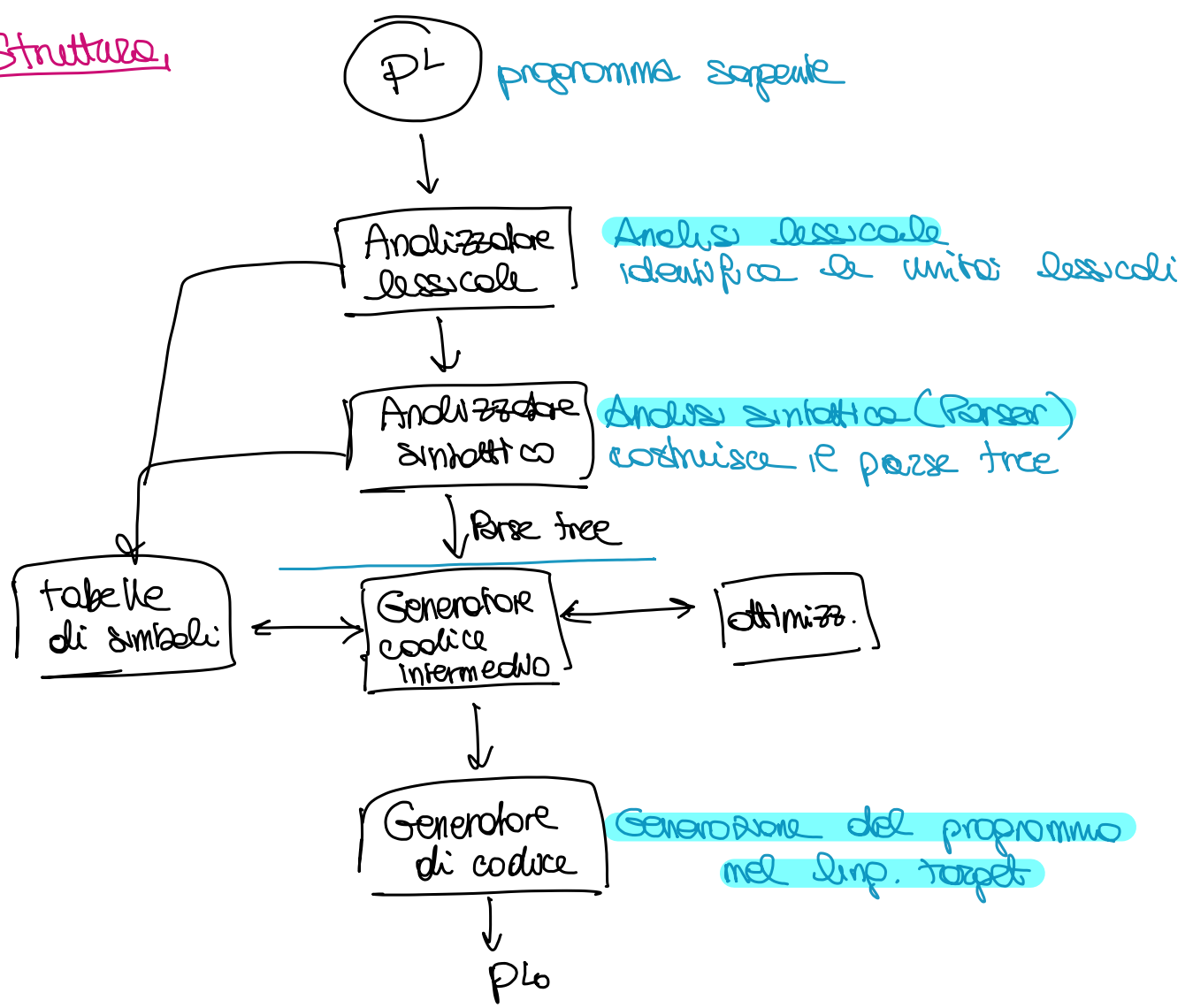
Equivalenza

P^L e P^{L_0} sono semanticamente

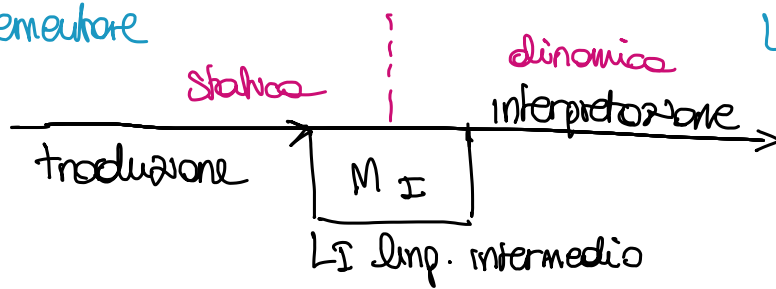
equivalenti



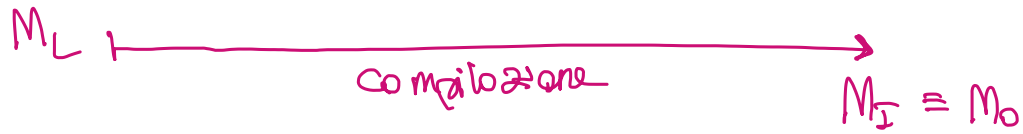
Struttura



L da implementare



Lo è il linguaggio implementato (e di solito)



Spec olizzatore

$$\text{Spec}_L : (\text{Prop}^L \times \text{Dato}) \rightarrow \text{Prop}^L$$

$$\text{Spec}_L(P^L, d) = P_d^L$$

← porzione dell'input

$$\forall y. P^L(y) = P_d^L(y)$$

← restante porzione

th. smn

Speciolizzare un interprete su un programma

→ un compilatore
otteniamo

DESCRIVERE 1 PL

↳ SINTASSI → Regole di costruzione di frasi ben formate
⇒ descrivere relazioni tra simboli

↳ SEMANTICA → Attribuzione del significato ai simboli
⇒ Data una frase che rispetta le regole sintattiche gli attribuisce un significato

↳ PRAGMATICA → Regole (non obbligatorie) di buon utilizzo (ingegnerato del SW)

↳ IMPLEMENTAZIONE → Come si espongono frasi ben formate che hanno significato

Sintassi → descrivere le regole di formazione

Parole stringhe caratteri

Frase sequenza di parole ben formate

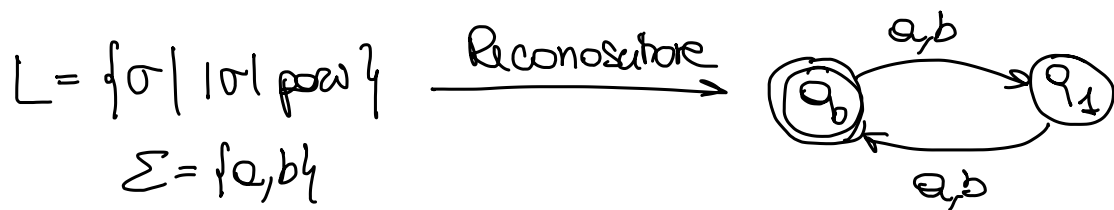
Linguaggio insieme delle frasi

Lessema parola con significato specifico (simboli terminali della grammatica)
unità sintattica del linguaggio

Token categoria sintattica (simboli non terminali)

↓
Riconoscitori
(Automatici)

↓
Generatore
(Grammatiche)



$G: S \rightarrow \epsilon \mid aS \mid bS \mid baS$

Descrizione della grammatica

→ Alfabeto (simboli)

→ Descrizione lessicale (sequenze di simboli che costruiscono parole del linguaggio → "simboli" terminali)

→ Regole sintattiche (Regole di derivazione della grammatica che compongono tra loro parole)

→ Categorie sintattiche ("simboli" non terminali delle token)

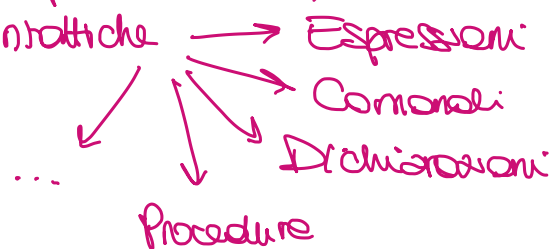
⇒ Grammatiche CF (BNF)

CFG $G = \langle V, T, P, S \rangle$

V insieme finito di simboli non terminali

elementi della frase (token)

categorie sintattiche



T insieme finito di simboli terminali

Vocabolario del linguaggio (lessemi)

while, if, :=, {, { ...
 + - ...

P produzioni $A \rightarrow \alpha \quad A \in V \quad \alpha \in (V \cup T)^*$

Regole di formazione del programma

Com $\rightarrow \dots$ while Exp do Com $\dots$

Exp $\dots$

SEV simbolo iniziale

Categorie dei programmi

Esempio

Exp

$\underbrace{\quad}_V \quad \underbrace{\quad}_T$
 $\langle \{E\}, \{ \text{or, and, not, (,)}, 0, 1 \}, (P) E \rangle$

$\rightarrow \underline{E} \rightarrow 0 \mid 1 \mid (\underline{E} \text{ or } \underline{E}) \mid$
 $(\underline{E} \text{ and } \underline{E}) \mid (\text{not } E)$